

Mammon V3 PRD - Part 2: Rust Execution Engine (The Sniper)

1. Engine Initialization & State

Target: execution_node/src/sniper_hft.rs

The Rust engine contains ZERO continuous math. It is a deterministic state machine controlled by variables injected via Redis.

```
pub struct SniperEngine {
    pub symbol: String,
    pub tick_size: f64,
    pub lot_step: f64,

    // Core Execution State
    pub aep: f64,           // Average Entry Price
    pub inventory_coin: f64, // Physical balance
    pub tfi_window: [f64; 100], // Ring buffer of Notional USD trade flow
    pub tfi_idx: usize,

    // AI Sniper Levers (Injected via Redis)
    pub live_momentum_trigger_obi: f64,
    pub live_take_profit_ticks: u32,
    pub live_stop_loss_ticks: u32,
    pub live_max_tranches: u32,

    // Action Directives
    pub open_bid: Option<f64>,
    pub open_ask: Option<f64>,
    pub emergency_dump_triggered: bool,
}
```

2. The Execution Loop (Tick)

Target: execution_node/src/sniper_hft.rs

On every LOB update, execute this strict sequence:

```
pub fn tick(&mut self, best_bid: f64, bid_vol: f64, best_ask: f64, ask_vol: f64) {
    let current_notional = self.inventory_coin * best_bid;
```

```

let active_tranches = (current_notional / 6.00).floor() as u32;
let obi = (bid_vol - ask_vol) / (bid_vol + ask_vol);

self.emergency_dump_triggered = false;
self.open_bid = None;
self.open_ask = None;

// 1. DUST RECOVERY: Trapped below $5.10 min notional
if self.inventory_coin > 0.0 && current_notional < 5.10 {
    self.open_bid = Some(best_bid); // Aggressively bid to clear dust
    return;
}

// 2. ENTRY: The Momentum Penny-Jump
if active_tranches < self.live_max_tranches {
    // Trigger condition: Massive Buy Wall
    if obi > self.live_momentum_trigger_obi {
        let jump_bid = best_bid + self.tick_size;

        // AEP DEFENSE: Mathematically forbid averaging UP into a pump.
        if active_tranches > 0 && self.aep > 0.0 && jump_bid >= self.aep {
            self.open_bid = None;
        } else {
            self.open_bid = Some(jump_bid);
        }
    }
}

// 3. EXIT: The Profit Strike
if active_tranches > 0 && self.aep > 0.0 {
    let target_price = self.aep + (self.tick_size * self.live_take_profit_ticks as f64);
    self.open_ask = Some(target_price);
}

// 4. FALLBACK: The Hard Stop-Loss (Amputation)
if active_tranches > 0 && self.aep > 0.0 {
    let stop_price = self.aep - (self.tick_size * self.live_stop_loss_ticks as f64);
    if best_bid <= stop_price {
        self.emergency_dump_triggered = true;
        self.open_ask = None;
        self.open_bid = None;
    }
}

```

}

3. Order Manager & User Data Stream (UDS)

Target: execution_node/src/main.rs & execution_node/src/binance_v3.rs

1. **UDS Integration:** Rust MUST connect to wss://stream.binance.com:9443/ws/<listenKey>.
2. **Zero-Latency State Update:** When an executionReport arrives confirming FILLED:
 - Immediately update engine.inventory_coin and engine.aep in memory.
 - Insert trade telemetry to PostgreSQL instantly via a background task.
3. **Strict Post-Only:** In binance_v3.rs, place_limit_order MUST enforce type=LIMIT_MAKER and selfTradePreventionMode=EXPIRE_MAKER.
4. **Panic Intercept:** Inside the WSS loop, if engine.emergency_dump_triggered == true, instantly cancel all open Asks and execute a place_market_sell for engine.inventory_coin.